

基于 PagedAttention 的大语言模型 高效内存管理与推理服务

Efficient Memory Management for Large Language Model Serving with PagedAttention
Woosuk Kwon¹ Zhuohan Li¹ Siyuan Zhuang¹ Ying Sheng^{1,2} Lianmin Zheng¹ Cody Hao Yu³
Joseph E. Gonzalez¹ Hao Zhang⁴ Ion Stoica¹

¹UC Berkeley ²Stanford ³Independent ⁴UC San Diego | SOSP '23 | github.com/vllm-project/vllm

摘要

大语言模型(LLM)的高吞吐推理需要在同一批次中处理足够多的请求。然而, 现有系统面临严峻挑战: 每个请求的键值缓存(KV Cache)内存巨大且动态增长和收缩。当管理不善时, 内存碎片化和冗余复制会严重浪费内存, 从而限制批次大小。为解决此问题, 我们提出 PagedAttention, 一种受操作系统虚拟内存和分页技术启发的注意力算法。在此基础上, 我们构建了 vLLM, 一个 LLM 推理服务系统, 实现了: (1) KV 缓存内存接近零浪费; (2) 请求内和请求间 KV 缓存的灵活共享, 进一步降低内存使用。评测表明, 相比 FasterTransformer 和 Orca 等最先进系统, vLLM 在相同延迟水平下将主流 LLM 吞吐量提升 2-4 倍。在更长序列、更大模型和更复杂解码算法下, 提升更为显著。

1. 引言

大语言模型(LLM)的涌现——如 GPT 和 PaLM——催生了编程助手和通用聊天机器人等新型应用, 正深刻影响我们的工作和日常生活。许多云服务商竞相提供这些应用的托管服务。然而, 运行这些应用成本极高, 需要大量 GPU 等硬件加速器。据近期估算, 处理一个 LLM 请求的成本可能是传统关键词查询的 10 倍。面对如此高昂的成本, 提高吞吐量——从而降低单次请求成本——变得愈发重要。

LLM 的核心是自回归 Transformer 模型。该模型基于输入(提示)和已生成的输出 token 序列, 逐个生成 token。对每个请求, 这一昂贵过程反复执行直至输出终止 token。这种顺序生成使工作负载呈现显存密集型特征, GPU 计算能力未被充分利用, 限制了推理吞吐量。

通过将多个请求打包到同一批次中可以提升吞吐量。但要在一个批次中处理大量请求, 每个请求的内存空间必须得到高效管理。以 13B 参数 LLM 在 NVIDIA A100 GPU (40GB) 上为例, 约 65% 内存被模型参数占用, 约 30% 用于 KV 缓存。KV 缓存是 Transformer 注意力机制的核心数据结构, 存储每个 token 的键值张量, 避免在后续解码步骤中重复计算。

由于传统深度学习框架主要针对训练而非推理场景设计, 现有 LLM 服务系统将 KV 缓存存储为跨请求不连续分配的连续张量, 导致: (1) 内部碎片——为预留未来 token 位置而过度分配; (2) 外部碎片——因各请求序列长度各异, GPU 显存产生无法利用的空隙。更糟糕的是, 现有系统将 KV 缓存按请求独立管理, 无法利用不同请求间的共享机会, 造成大量内存浪费。

本文提出 PagedAttention, 一种受操作系统分页机制启发的注意力算法。与传统注意力不同, PagedAttention 允许 KV 缓存存储在非连续、固定大小的「页面」(blocks)中, 就像虚拟内存中的页面。基于此, 我们构建了 vLLM, 一个 LLM 推理服务系统, 其核心思想: 操作系统中的虚拟内存管理思想——页面映射、按需分配、写时复制——同样适用于 LLM 的 KV 缓存。vLLM 实现: (1) KV 缓存近似零内存浪费; (2) 通过块级共享实现请求内和请求间的 KV 缓存复用; (3) 高效的块级内存管理。

本文主要贡献: (a) 识别了 LLM 推理服务中 KV 缓存内存浪费的根本原因; (b) 提出 PagedAttention, 一种在非连续分页内存上操作的新型注意力算法; (c) 设计并实现了 vLLM, 基于 PagedAttention 构建的分布式 LLM 推理引擎; (d) 在多种模型和工作负载上评测 vLLM, 展示了相比现有系统的显著性能提升。

2. 背景

2.1 基于 Transformer 的大语言模型

语言建模的任务是建模 token 序列 $(x_1, \dots, x_n)$ 的概率。由于语言具有天然的顺序性, 通常将联合概率分解为条件概率的乘积(即自回归分解):

$$P(x) = P(x_1) \cdot P(x_2|x_1) \cdot \dots \cdot P(x_n|x_1, \dots, x_{n-1}) \quad (1)$$

Transformer 已成为这种概率建模的事实标准架构。注意力机制是其核心操作。给定输入隐藏状态 $X \in \mathbb{R}^{n \times d}$ (n 个 token, 维度 d), 注意力计算为:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V \quad (2)$$

其中 Q (查询)、 K (键)、 V (值)由 X 与各自权重矩阵相乘得到。在自回归解码中, 每个 token 只能关注其前置 token, 使用因果掩码(causal mask)。

2.2 LLM 推理服务

一次推理通常分为两个阶段: 预填充阶段(Prefill)——将整个用户提示并行输入模型, 生成第一个输出 token 并计算 KV 缓存, 该阶段 GPU 并行计算利用率高; 解码阶段(Decode)——每步生成一个新的输出 token, 同时使用并更新 KV 缓存, 该阶段顺序执行且瓶颈在于 KV 缓存的读写速度(显存带宽)。

服务系统通常采用连续批处理(continuous batching), 即在解码过程中动态添加新请求到当前批次, 充分利用模型的空闲算力。然而, 这加剧了对 KV 缓存高效管理的需求, 因为每个批次中不同请求的序列长度各不相同。

2.3 KV 缓存的内存挑战

在自回归解码过程中, 所有 token 的键和值张量需保存以避免重复计算。这些保存在 GPU 显存中的张量统称为 KV 缓存。对于 L 层的 Transformer 模型, 每层都需要独立的 K 和 V 张量。假设使用 bfloat16(2 字节/元素), 一个请求的 KV 缓存约为:

$$\text{KV Cache Size} = 2 \times L \times 2 \times n \times d \times 2 \text{ (bytes)} \quad (3)$$

例如, 13B OPT 模型生成 1K token 输出, KV 缓存约需 4.3 GB。而且随着模型规模增大和序列变长, KV 缓存会迅速膨胀。当多个请求并发时, KV 缓存总量轻易超出 GPU 显存, 成为制约批次大小和服务吞吐量的主要瓶颈。

3. LLM 推理中的内存挑战

尽管 KV 缓存占据大量 GPU 显存, 现有系统却以极其低效的方式管理这些内存。主要有三大浪费来源:

3.1 为未来 token 预留内存(内部碎片)

现有系统在请求开始时便为其分配一块连续的 KV 缓存内存, 大小按最大可能序列长度计算。例如, 若最大序列长度为 2048, 系统立即预留所有 2048 个 token 位置的 KV 缓存——尽管实际生成的序列可能远短于此。这种做法造成严重的内部碎片: 预留但从未使用的内存被白白浪费。

3.2 跨请求的歧异序列长度(外部碎片)

即使采用按需分配策略, 因解码过程中各请求的内存需求动态变化。当某请求完成释放内存后, 其释放的空间可能无法被另一请求使用(需要更大的连续块), 形成外部碎片。在典型工作负载下, 这些浪费合计可达有效内存的 40-80%。

3.3 冗余复制

并行采样(parallel sampling)、束搜索(beam search)、推测解码(speculative decoding)等高级解码算法会在同一请求内生成多个输出序列。在现有系统中, 这些序列通常共享提示部分, 但提示的 KV 缓存却被为每个序列独立复制。这不仅浪费大量内存(提示通常占序列的很大比例), 也成为实际启用这些算法的障碍。

4. PagedAttention

为解决上述问题, 我们提出 PagedAttention。核心洞察是: KV 缓存不需要存储在连续内存中。借鉴操作系统虚拟内存中的分页技术, PagedAttention 将每个序列的 KV 缓存划分为固定大小的 KV 块(blocks), 每个块包含固定数量 token 的键和值向量。因此, 注意力计算被重新表述为逐块(block-wise)的方式, 而非逐 token 的方式。

4.1 分块注意力计算

形式上, 设每个 KV 块包含 B 个 token。对于查询向量 q (来自当前 token), PagedAttention 的计算过程为: 对每个非连续的 KV 块, 分别计算该块内所有 token 的点积得分, 再进行 softmax, 最后对各块的 V

矩阵加权求和并拼接。这在数学上等价于标准注意力, 因为 softmax 可以按块分解。

实现上, PagedAttention 将块表(block table)——记录每个序列的逻辑块到物理块的映射——作为 GPU 内核的输入。GPU 内核通过块表查找物理块地址, 异步并行地计算各块的注意力, 最终组合成完整输出。关键优势: 非连续内存布局使得物理块可以跨请求自由复用, 就像操作系统中的物理页框。

4.2 KV 缓存管理

当接收到新请求时, vLLM 不预先为其分配最大长度的连续内存, 而是仅在需要时才分配 KV 块。每生成 B 个 token, 就从块池中分配一个新块。当请求完成时, 其占用的所有块被释放回池中, 可立即被新请求使用。这种按需分配 + 即时回收机制从根本上消除了内部碎片, 并将外部碎片降至最低——因为所有块大小一致, 任意空闲块都可分配给任意请求。

vLLM 的块管理器(BlockManager)维护一个统一的块池, 支持以下核心操作: 分配(从空闲列表取出一块)、释放(将块归还到空闲列表)、引用计数(跟踪有多少序列共享同一块)、写时复制(Copy-on-Write, CoW)——当需要修改被多个序列共享的块时, 先复制再修改, 这是实现高效内存共享的核心机制。

4.3 请求间与请求内的 KV 缓存共享

并行采样与束搜索: 同一提示的多个输出序列可以共享提示部分的 KV 块。vLLM 使用写时复制(CoW)机制: 当序列在产生第一个差异 token 之前, 它们引用相同的物理块。当某序列需要写入新 token 时, vLLM 先复制该块的引用, 再为新块写入数据。这样, 提示部分的 KV 缓存只需存储一份, 节省了 $W \times (\text{提示长度})$ 的内存, 其中 W 是序列数。

跨请求共享: 进一步的, 不同请求之间如果有相同的提示前缀, 也可共享对应的 KV 块。这在实际系统中非常重要, 因为许多对话场景(如聊天机器人)都包含共享的系统提示(system prompt)。vLLM 通过维护一个全局的前缀缓存(prefix cache), 自动检测和复用已有的 KV 块, 显著减少首次 token 时间(TTFT)。

5. vLLM 系统设计

vLLM 是一个端到端的 LLM 推理服务系统, 采用中心化调度器(centralized scheduler)架构。系统由以下核心组件构成: (1) 前端(Frontend): 接收 HTTP/gRPC 请求, 将 token 序列传递给调度器; (2) 调度器(Scheduler): 核心控制组件, 维护请求队列、决定批次组合、管理块分配; (3) 块管理器(BlockManager): 管理 GPU 和 CPU 上的 KV 块池, 实现分配、释放、CoW 和交换; (4) GPU 执行器(Worker): 封装模型推理引擎, 执行 PagedAttention 内核; (5) 分布式运行时: 支持张量并行(tensor parallelism), 在多个 GPU 上横向扩展模型层。

5.1 调度策略

vLLM 的调度器采用先来先服务(FCFS)策略处理请求, 并在每个解码步骤维护一个活跃批次。当批次有空位(块内存可用且 GPU 算力未饱和), 调度器从等待队列中拉取新请求加入。核心约束:

显存约束: 批次中所有请求的 KV 块总数不得超过块池大小。这保证系统永远不会 OOM。

公平性: vLLM 限制单个请求在单次调度中最多处理的 token 数(max_num_batched_tokens), 防止单个长请求「饿死」短请求。

此外, vLLM 支持抢占(preemption): 当显存紧张时, 调度器可将低优先级请求的 KV 块交换(swap)到 CPU 内存, 腾出 GPU 空间给活跃请求; 待 GPU 空闲后再换回。这是操作系统虚拟内存中交换(swapping)思想的直接映射。

5.2 分布式执行

对于超出单 GPU 显存的大模型, vLLM 支持张量并行(tensor parallelism)。每层 Transformer 的注意力头在多个 GPU 上分片, 块管理器通过 NCCL 在各 GPU 之间同步块元数据(逻辑→物理映射), 保证 PagedAttention 内核在各 GPU 上能独立、一致地访问 KV 块。PagedAttention 的分块特性使分布式执行更加自然: 因为各 GPU 的块表一致, 只需同步元数据(相比传输整个 KV 缓存), 通信开销极低。

6. 评测

我们在多种设置下对 vLLM 进行了广泛评测, 与三个最先进的系统对比: FasterTransformer(NVIDIA 官方推理后端)、Orca(学术界的连续批处理系统)和 DeepSpeed-MII。

6.1 实验设置

模型: OPT-13B、OPT-66B、LLaMA-13B、LLaMA-65B。所有模型均以半精度(FP16)运行。

硬件: 13B 模型使用 1 张 NVIDIA A100-40GB GPU; 大模型使用 4 张 A100-40GB(张量并行)。

工作负载: 两个真实数据集——(1) ShareGPT 数据集(以长对话为主), (2) Alpaca 数据集(指令遵循任务, 较短)。
关键指标: 吞吐量(token/s)和延迟(P50/P99 每 token 时间)。

6.2 基线性能比较

在 ShareGPT 数据集上, vLLM 的吞吐量显著优于所有基线。具体地, 相比于 Orca(Max), vLLM 将 OPT-13B 的吞吐量提升了 2.0 倍; 相比于 FasterTransformer, 提升达 3.9 倍。对于更大的模型(LLaMA-65B), 增益更加明显——相比于 Orca(Pow2)提升 2.3 倍。在 Alpaca 数据集(相对较短的指令)上, vLLM 同样保持领先, 比 Orca(Max)平均提升约 1.5 倍吞吐。

6.3 内存效率分析

为量化内存效率, 我们测量了不同系统在稳定状态下 KV 缓存内存的浪费比例。Orca(Max)的浪费最高, 高达 57.3% 的 KV 缓存内存未被有效利用(主要为内部碎片); Orca(Pow2)浪费约为 41.6%。相比之下, vLLM 的浪费率仅为 3.5%, 接近理论最优。这主要归功于分页机制——按需分配消除了内部碎片, 统一块大小将外部碎片降至最低。

6.4 共享机制的效果

我们通过并行采样($\text{beam search width} = 4$)评估了 KV 缓存共享机制的效果。vLLM 的吞吐量随采样宽度的增加几乎保持不变, 而 Orca 的吞吐量随采样宽度增加而显著下降(采样宽度 = 4 时吞吐量降至基线约 30%)。这是因为 vLLM 通过 CoW 机制共享了提示部分的 KV 缓存(约占总量的 60-80%), 而 Orca 为每个序列完整复制。

6.5 交换与抢占

vLLM 的 CPU-GPU 块交换机制在显存紧张时表现出良好弹性。当请求速率突增导致 GPU 显存接近饱和时, vLLM 自动将部分等待最久的请求的 KV 块换出到 CPU, 保证新请求能正常加入批次。虽然交换操作引入一定延迟膨胀(约 20-30% 的 P99 延迟增加), 但系统保持稳定不崩溃, 而 Orca 在同等负荷下因 OOM 直接失败。

7. 扩展: 预填充与解码的解耦

PagedAttention 的块抽象不仅优化了内存管理, 还自然地支持将预填充和解码阶段解耦到不同的 GPU 上。预填充是计算密集型(大量 token 并行计算注意力), 解码是显存密集型(每个新 token 只做少量计算但大量读写 KV 缓存)。解耦后, 预填充 GPU 可以使用高算力 GPU(如 H100), 解码 GPU 可以使用大显存 GPU(如 A100), 实现异构资源的最优利用。这进一步提升了整体吞吐量和硬件利用率。

8. 相关工作

LLM 推理优化: 多项工作致力于优化 Transformer 推理性能。FasterTransformer 提供高度优化的 GPU 内核, 但缺乏动态内存管理。Orca 引入迭代级调度允许在解码过程中动态更新批次, 但内存管理仍采用连续预留方案。DeepSpeed-Inference 关注大规模模型的多 GPU 推理, 同样沿用连续 KV 缓存。LightSeq、TurboTransformers 等通过内核融合和量化加速推理, 但未解决 KV 缓存的内存碎片问题。

高效内存管理: 操作系统和编程语言运行时的内存管理(分页、分段、垃圾回收)为 vLLM 提供了灵感。FlashAttention 通过 IO-aware 的块式计算优化注意力, 但在内存管理层面仍使用连续张量。FlexGen 探索了在有限 GPU 显存下的离线批量推理, 但不针对在线服务场景。

高级解码算法: 推测解码、并行解码等方法通过生成多个候选 token 来加速推理。然而, 这些算法的实际收益严重依赖高效的 KV 缓存共享——这正是现有系统缺乏而 vLLM 提供的核心能力。

9. 讨论与结论

本文提出了 PagedAttention 和 vLLM 系统, 将操作系统虚拟内存的分页思想应用于 LLM 推理服务中的 KV 缓存管理。通过将 KV 缓存划分为固定大小的块并使用块表实现逻辑到物理的映射, vLLM 实现了接近零的内存浪费和灵活的 KV 缓存共享。实验表明, vLLM 在保持低延迟的同时, 将主流 LLM 的吞吐量提升 2-4 倍, 尤其在长序列、大模型和复杂解码场景下优势更为突出。

局限性

首先, PagedAttention 引入了额外的间接层(块表查找), 可能带来轻微的计算开销。不过, 我们定制的 GPU 内核将此开销控制在 5% 以内。其次, 当前 vLLM 的实现主要针对 NVIDIA GPU, 对其他硬件平台的适配(如 AMD、Apple Silicon)仍在进行中。最后, 系统在极高负载(如 100K token 超长上下文)下的块交换策略仍需进一步优化。

未来方向

多个有前景的方向值得探索: (1) 结合量化(quantization)技术进一步压缩 KV 缓存大小; (2) 更智能的调度策略——基于预测请求长度而非简单的 FCFS; (3) 将 PagedAttention 的思想推广到训练场景, 管理大规模训练的激活(activation)内存; (4) 探索页面的最优大小和混合精度(mixed-precision)KV 缓存。

vLLM 已在 GitHub 开源(github.com/vllm-project/vllm), 自发布以来获得了社区的广泛关注和贡献。我们相信, 将系统设计原则(尤其是虚拟内存)应用于机器学习基础设施是一个有前途的方向, PagedAttention 只是其中一例。更多的「系统 + ML」交叉创新值得期待。

致谢

感谢匿名审稿人的宝贵反馈。感谢 LMSYS 组织提供 Chatbot Arena 平台支持的讨论。本研究部分由 NSF、IBM、Google、Apple 和各大云服务商资助。

参考文献(精选)

- [1] Josh Achiam et al. GPT-4 Technical Report. arXiv:2303.08774, 2023.
- [2] Reza Yazdani Aminabadi et al. DeepSpeed-Inference. arXiv:2207.00032, 2022.
- [3] Tom Brown et al. Language Models are Few-Shot Learners. NeurIPS, 2020.
- [4] Mark Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
- [5] Aakanksha Chowdhery et al. PaLM: Scaling Language Modeling with Pathways. JMLR, 2023.
- [6] Tri Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention. NeurIPS, 2022.
- [7] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism. arXiv:2307.08691, 2023.
- [8] Hugo Touvron et al. LLaMA: Open Foundation Models. arXiv:2302.13971, 2023.
- [9] Ashish Vaswani et al. Attention Is All You Need. NeurIPS, 2017.
- [10] Susan Zhang et al. OPT: Open Pre-trained Transformer LMs. arXiv:2205.01068, 2022.
- [11] Gyeong-In Yu et al. Orca: A Distributed Serving System. OSDI, 2022.
- [12] Ying Sheng et al. FlexGen: High-Throughput Generative Inference. ICML, 2023.
- [13] Yaniv Leviathan et al. Fast Inference via Speculative Decoding. ICML, 2023.
- [14] NVIDIA. FasterTransformer. github.com/NVIDIA/FasterTransformer, 2023.
- [15] DeepSpeed Team. DeepSpeed. github.com/microsoft/DeepSpeed, 2022.
- [16] Sebastian Bubeck et al. Sparks of AGI. arXiv:2303.12712, 2023.